

Trekking Management App Project Report - Modern Application Development-I

Student Details

- Name - Soumya Ranjan Panda
- Roll Number - 23F2004742
- Email - 23f2004742@ds.study.iitm.ac.in

Description

Trekking Management App is a role-based Flask web application developed to manage trekking routes, bookings, staff assignment, participant tracking, and user trekking history. It supports Admin, Staff, and Trekker roles with separate permissions. Admin users control the platform, staff members manage only assigned treks, and trekkers browse open treks, book available slots, cancel bookings, update profiles, and view history. The application emphasizes secure authentication, staff approval, slot validation, and simple API-based data access.

Key capabilities include:

- Role-based authentication and protected dashboards
- Admin trek creation, editing, deletion, and staff assignment
- Staff registration with approval, blacklist, and restore workflow
- Trekker registration, profile update, booking, cancellation, and history
- Availability and slot tracking for every trek
- Participant lists for staff and booking overview for admins
- Admin search across treks, users, and staff
- JSON APIs support for treks, bookings, and users

Frameworks and Libraries Used

Backend

- Python 3
- Flask 3.0.3 web framework
- Flask-Login for authentication and sessions
- Flask-SQLAlchemy for ORM database operations
- SQLite database
- Werkzeug for password hashing

Frontend

- HTML, CSS, and Bootstrap-based responsive layout
- Jinja2 template rendering
- Custom static styling for dashboards and forms

Database Schema

The database design follows a relational model to store users, staff profiles, trek records, and booking history.

User Table

- id, name, email, phone, password_hash
- role: admin/staff/user
- status, is_approved, created_at

Trek Table

- trek_name, location, difficulty, duration_days
- total_slots, available_slots, description
- start_date, end_date, status, assigned_staff_id

Booking Table

- user_id and trek_id foreign keys
- booking_date, status, payment_status, updated_at
- unique user-trek constraint to avoid duplicate entries

StaffProfile Table

- user_id foreign key
- experience_years, specialization, emergency_contact, bio
- created_at and updated_at timestamps

Approach Used

Role-Based Access Control (RBAC)

- Admin manages system data, staff approvals, users, bookings, and trek assignment.
- Staff can access only approved assigned treks and participant details.
- Trekkers browse open treks, book slots, cancel bookings, and maintain profiles.

Authentication

- Flask-Login handles login, logout, sessions, and protected routes.
- Werkzeug password hashing stores credentials securely.
- Blacklisted or unapproved staff accounts are blocked from protected workflows.

Validation and Business Logic

- Trek forms validate required fields, numeric ranges, difficulty/status values, and date order.
- Bookings are allowed only for open treks with available slots.
- Cancelling a booking restores slot availability and updates booking status.
- Staff assignment is restricted to approved active staff members.

Features and Functionalities

User Authentication

- Login and logout for all roles
- Trekker and staff registration
- Role-specific dashboard redirection

Admin Management

- Dashboard counts for treks, users, staff, and bookings
- Create, edit, assign, and delete treks
- Approve, blacklist, restore staff, and manage trekker accounts
- Search and review all bookings

Staff Management

- Assigned trek dashboard
- Update trek status and available slots
- View participant details for assigned treks
- Maintain experience, specialization, emergency contact, and bio

Trekker Booking

- Browse and filter open treks by difficulty and location
- View trek details and book available slots
- Cancel active bookings and view complete trekking history

Application Architecture

The project uses a compact Flask architecture with SQLAlchemy models, route-level access control, Jinja2 templates, static CSS, and SQLite persistence. The backend defines validation helpers, authentication decorators, role-specific web routes, and REST-style JSON endpoints. The frontend contains templates for landing, login, dashboards, trek forms, profile, history, and management screens. The database is initialized on startup with schema checks and a seeded

admin account.

Project Structure

- Trekking-Management-App-V1/
- app.py - Flask routes, validation, API endpoints, and app setup
- model.py - SQLAlchemy models for User, Trek, Booking, and StaffProfile
- templates/ - Jinja2 pages for admin, staff, trekker, auth, and trek views
- static/css/style.css - custom application styling
- requirements.txt - Python package dependencies

Presentation Video

[Trekking Management Application, MAD-1](#)